A grid world agent with favorable inductive biases

Anonymous submission

Abstract

We present a novel experiential learning agent with causally-informed intrinsic reward that is capable of learning sequential and causal dependencies in a robust and data-efficient way within grid world environments. After reflecting on state-of-the-art Deep Reinforcement Learning algorithms, we provide a relevant discussion of common techniques as well as our own systematic comparison within multiple grid world environments. Additionally, we investigate the conditions and mechanisms leading to data-efficient learning and analyze relevant inductive biases that our agent utilizes to effectively learn causal knowledge and to plan for rewarding future states of greatest expected return.

1 Introduction

Grid world environments come in many forms and have been studied extensively in the history of Artificial Intelligence with some notable examples such as Wumpus World (Bryce 2011), Minigrid (Chevalier-Boisvert et al. 2024), and Tileworld (Pollack and Ringuette 1990). However, the creation of intelligent grid world agents capable of learning effectively and in a data-efficient way has posed significant challenges. Current Reinforcement Learning (RL) agents struggle in some instances due to sequential dependencies, partial observability, relatively high-dimensional state spaces, compared to more traditional RL tasks (even when a single value per cell is provided rather than per pixel), and sometimes non-deterministic effects of actions.

Sequential dependencies usually raise the training data demand exponentially depending on the combinatorics, and ultimately the number of the arising options, unless the agent is capable of learning causal representations that transfer well because chaining them is favorable for reaching an intended outcome. Partial observability, on the other hand, may require the model to have access to information from prior states, which typically corresponds to the previously observed values in a grid world outside the agent's current field of view. In terms of input dimensionality, there is a trade-off between observation window being too small for learning an effective policy and the agent's observation window being more high-dimensional thereby demanding more training data, even though this might be the least problematic. Additionally, if the agent is able to represent values outside of its observation window, a learned policy needs to consider not only the observation window itself but also how it spatially relates to the remembered information beyond it.

With these considerations in mind, we introduce Non-Axiomatic Causal Explorer (NACE), a novel experiential learning agent, an extension of prior work (Cook and Hammer 2024), which possesses learning mechanisms with the involved inductive biases. NACE is designed to induce causal rules from temporal and spatial local changes in the grid, which are often (but not always) caused by the agent. It utilizes these rules to plan for and reach future states of maximum uncertainty in order to effectively learn more (causal rules) about the environment.

To illustrate the effectiveness of our approach, we provide a comprehensive discussion of state-of-the-art Deep Reinforcement Learning (DRL) techniques as well as our own systematic comparison within multiple grid world environments. We thoroughly investigate the conditions and mechanisms under which learning in grid world environments can become more data-efficient and attempt to answer the question about which inductive biases can lead to close-to-optimal learning speeds in the case where the agent is not pre-equipped with particular interaction rules between grid cell types but has the inductive biases to build and use them effectively. Lastly, we analyze useful inductive biases applicable among a wide range of grid worlds and their generality to raise data efficiency of learning in such domains.

2 Related Works

Current RL techniques such as value-based (Deep Q-Learning (Mnih et al. 2013)) and policy gradient based (Proximal Policy Optimization (Schulman et al. 2017) and variants) typically demand millions of training iterations to deal with grid world environments (Zhang et al. 2020).

Attempts have been made to improve exploration with intrinsic rewards, for instance by taking information gain into account. However, these are typically based on simple proxies, such as based on prediction errors (Burda et al. 2018) or how often states have been visited (or specific action has been taken on it (Zheng et al. 2021)). We take a different route and propose causally-informed exploration which considers whether learned cause-effect relations are applicable (their preconditions match) to relevant parts of the state.

Practical means-end reasoning has been extensively studied in symbolic AI, with various planning systems such

as STRIPS planners, which are also used in robotics (Guo et al. 2023). While these systems can effectively deal with human-given causal knowledge in logical form, they are not equipped with structured learning abilities. The same is true about Behavior Trees, which are a powerful and transparent tool to encode human hierarchical procedural knowledge in a computer system (Colledanchise and Ögren 2018).

Partially Observable Markov Decision Processes (POMDP) (Spaan 2012) is another practical approach to let RL agents consider high-level causal knowledge given and known by humans. However, most work on POMDP has focused on the updating of conditional probability values, rather than the discovery of new causal relations.

Judea Pearl has proposed causal networks (Pearl 1995), which allow for the deriving of new probability spaces from existing ones and performed interventions, where random variables get fixated on specific values. Pearl's approach is deductive-only in nature and demands a causal graph to exist before new probability spaces can be derived from interventions. A work has been proposed permitting for the structure learning of causal relations, more traditionally as a form of combinatoric optimization problem and more recently with continuous optimization problem with special constraints assuring a Directed Acyclic Graph (Zheng et al. 2018) will result whenever a solution exists. However, for a specific set of collected data points, the direction of a supposed causal relation is often fundamentally ambiguous, which often leads to erroneous results, especially when limited data is available. Finally, the causal graph cannot be easily updated when new data points become available, at least not without repeating the optimization process with the available data. While we make use of causal relations too, we aim for a real-time learning approach that is sufficient for agents in grid worlds.

3 Attempted methods in grid worlds

3.1 Reinforcement Learning

Reinforcement Learning techniques face a significant challenge of sample efficiency, that is the ability to learn effective policies from a few interactions with the environment. In this paper, we consider the current SOTA techniques extracted from the literature, as well as four fundamental algorithms: Deep Q-Network (DQN) (Mnih et al. 2015), Proximal Policy Optimization (PPO) (Schulman et al. 2017), Advantage Actor-Critic (A2C) (Mnih et al. 2016), and Trust Region Policy Optimization (TRPO) (Schulman et al. 2015). Although each algorithm has its own strategies for sample efficiency, their limitations lead them to perform suboptimally in general complex scenarios.

1. **DQN** combines the power of deep neural networks with classical Q-learning to effectively handle large state spaces in high dimensions (Mnih et al. 2015). DQN is an off-policy algorithm that implements experience replay and a separate target network to mitigate the correlation between consecutive updates and stabilize the learning process. This off-policy workaround enables DQN to learn from its older experiences stored in the replay buffer, and hence, may improve utilization of data. Still, it suffers from overestimation bias and becomes very unstable in settings with sparse rewards, severely restricting its sample efficiency.
2. **A2C** is a synchronous deterministic variant of an actor-critic method, A3C, which reduces the high variance issues seen with the A3C algorithm. Yet, it is keeping the focus on parallel training by using multiple actors (Mnih et al. 2016). A2C tries to make better use of the samples by decreasing variance in policy updates using an advantage function estimation with adapted networks for policies (actors) and values (critic). However, its on-policy characteristic requires an entire roll-out to interact with the environment in every update (i.e., no data reuse), posing a hurdle towards high sample efficiency.
3. **TRPO** is developed to provide a more stable and reliable update mechanism in policy optimization (Schulman et al. 2015), ensuring large policy updates do not result in performance collapse by using a trust region constraint. TRPO employs a KL-divergence constraint to manage the extent of change in the policy distribution, ensuring incremental but consistent improvements. Although this method offers a theoretically grounded approach to mitigating policy collapse, its reliance on second-order optimization techniques incurs substantial computational overhead, detracting from its practical sample efficiency. Moreover, similarly to A2C, TRPO's on-policy nature necessitates fresh samples for each update cycle, leading to inefficient data usage.
4. **PPO** incorporates the multi-worker architecture found in A2C and adapts the trust region concept from TRPO to modulate the policy updates. With this, PPO efficiently balances computational complexity with performance (Schulman et al. 2017). Specifically, by using a clipped surrogate objective, PPO limits the size of policy updates to maintain training stability and employs multiple epochs of stochastic gradient descent per data batch, thereby improving upon the data utilization inefficiencies of its predecessors. Nevertheless, PPO's dependency on precise hyperparameter tuning and its on-policy framework, which limits data reuse, pose challenges to its sample efficiency in varied and complex environments.
5. **Intrinsic reward:** We have selected recent methods (Chen and Guan 2023) (Zhang et al. 2021) (Guo et al. 2022) featuring intrinsic rewards which aim to provide models of curiosity for more efficient exploration. For these we summarize and cite existing average reward values from the literature in a later section.

3.2 Experiential Learning

NACE is our novel experiential learning technique with causality-informed intrinsic reward and inductive biases for grid world environments to boost sample efficiency further. We will comprehensively introduced it in the next section.

Additionally, we used the Behavior Tree (BT) approach to have a closer-to-optimal upper bound policy to compare within the non-stationary environments, as well as a hard-coded optimal policy for the static ones.

4 Non-Axiomatic Causal Explorer

4.1 Inductive Biases

It is well-known that favorable inductive biases can help to boost the sample efficiency of Machine Learning models. However, the following are particularly inductive biases that are relevant for a large class of grid world environments:

1. **Temporal Locality**: Ability to model relevant dependencies locally in time, in simplest case just involving prior simulation frame, essentially Markov Assumption.
2. **Causal Representation**: Ability to model causal dependencies independently of acquired reward, making the model able to learn beliefs of the environment that can be chained and are agnostic to its objective.
3. **Spatial Equivariance**: Ability to model causal dependencies between grid cells independently of the specific location of the cells considered in the dependency.
4. **State Tracking**: Ability to effectively track state outside of the field of view of the agent based on the recorded or estimated locations.
5. **Attentional Bias** Relevant dependencies tend to involve values that have either observably changed or a different value than predicted.

4.2 Curiosity model

In this section, we will discuss the principles and mechanisms according to which the NACE agent can systematically acquire causal knowledge it lacks about the environment. The key principle is realized by making the agent plan to reach a state which it is most unfamiliar with. The familiarity is judged by whether existing rules match well to the situation, whereby matching is a matter of degree dependent on how many rule conditions match to the cells in the known state. This motivates the following formalism:

Match Quotient of a rule r is evaluated relative to cell c:

$$Q(r,c) = \frac{n_{matched\_conds}}{n_{conds}}$$

Cell match value of a Cell c dependent on all k existing rule match proportions:

$$C(c) = max(Q(r_1,c),\ldots,Q(r_k,c))$$

State match value of a predicted State s is the maximum match value of all m cells in the state:

$$S(s) = max(C(c_1),\ldots,C(c_m))$$

This value, as we will see, will be treated as the secondary objective in the planning process that guides the agent's decisions.

4.3 States and rule representation

State in NACE consists of a 2-dimensional array and a one-dimensional array. The 2-dimensional array reflects the spatial structure in the grid world (including the remembered cell outside of the current view of the agent), while the one-dimensional array has no notion of spatial distance and is for internal values such as inventory items, reward, etc.

Each rule is of the form $(precondition, action) \Rightarrow consequence$ whereby the precondition can hold a conjunction of cell values spatially relative to the cell value of the consequence, and the consequence predicts one particular cell's next value as well as the values of the one-dimensional array.

Each rule tracks evidence counters w_+ and w_- similar to (Wang 2013), whereby w_+ is increased whenever a perfectly matching rule predicted correctly, while w_- is increased whenever it predicts incorrectly.

4.4 Flow diagram

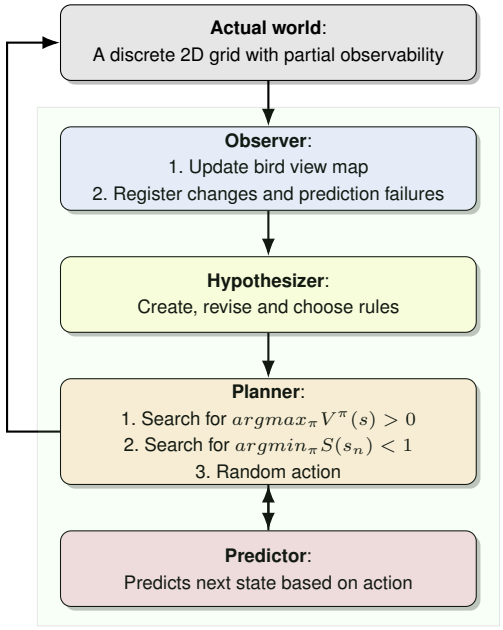

Figure 1: Flow diagram of the system

1. **Observer**: Update bird view map via values from partial observation array, then find changes in input, as well as prediction-observation mismatches (prediction failures). Formally this corresponds to determining the sets

$$M_{change_t} = \{c^{observation}_{t,x,y} | c^{observation}_{t,x,y} \neq c^{observation}_{t-1,x,y}\}$$

$$M^{observation}_{mismatched_t} = \{c^{observation}_{t,x,y} | c^{prediction}_{t,x,y} \neq c^{observation}_{t,x,y}\}$$

$$M^{prediction}_{mismatched_t} = \{c^{prediction}_{t,x,y} | c^{prediction}_{t,x,y} \neq c^{observation}_{t,x,y}\}$$

2. **Hypothesizer**: Associating positive and negative evidence based on prediction success, creating new rules rules when positive evidence is found for the first time.

   For each rule (with 2 precondition cells for Minigrid)

$$r = ((\overline{c}^1_{t-1} \wedge \ldots \wedge \overline{c}^k_{t-1} \wedge \overline{v}_{t-1} \wedge \overline{a}_{t-1}) \Rightarrow (\overline{c}_t \wedge \overline{v}_t \wedge R(r)))$$

   with reward $R(r) = R_t$ when the rule is created in time step t, if equality constraint $\overline{c}^1_{t-1}$ and $\overline{c}^2_{t-1}$ holds, meaning the precondition corresponds to the cell values in the previous known world state s_{t-1} and 1D array had value

according to $\overline{v}_{t-1}$, and $\overline{a}_{t-1}$ constrains the last executed action, then

$$M_t = (M_{change_t} \cup M^{observation}_{mismatched_t})$$

$$w_+(r) = \begin{cases} w_+(r) + 1 & \text{if } \{c^1_{t-1}, c^2_{t-1}, c^3_t\} \subseteq M_t \\ w_+(r) & \text{otherwise} \end{cases}$$

$$w_-(r) = \begin{cases} w_-(r) + 1 & \text{if } c^3_t \in M^{prediction}_{mismatched_t} \\ w_-(r) & \text{otherwise} \end{cases}$$

Finally, rules r for which $w_-(r) > w_+(r)$ are inactive, and for two rules r_1, r_2 if the preconditions match (including the action) but the postcondition differs, only the rule of the higher truth expectation is active, which is calculated in accordance with the following definitions:

$$w(r) = w_-(r) + w_+(r), f(r) = \frac{w_+(r)}{w(r)}, c(r) = \frac{w(r)}{w(r)+1}$$

$$f_{exp}(r) = (f(r) - \frac{1}{2}) * c(r) + \frac{1}{2}$$

This not only allows the system to find the relevant preconditions under which a consequence happens when the action is utilized, but also gives the system tolerance to non-deterministic effects, though analysis of the latter is beyond the scope of this paper.

3. **Planner**: NACE makes use of depth- and width-bounded Breadth-First-Search algorithm with a combined search objective consisting of two components: it searches for states resulting from the different action sequences for futures that lead to max. expected return or, if not existing, lowest state match value. Hence, it applies a key Reinforcement Learning principle to maximize the expected return, with the policy determined by the considered action sequence: $\pi(t) = a_t \quad \text{for} \quad t = 1, 2, \ldots, n$ whereby n is smaller-or-equal (dependent on where the optimum is found) than the maximum planning horizon:

$$argmax_\pi V^\pi(s_0) > 0$$

$$V^\pi(s) = \mathbb{E}\left[\sum_{t=0}^{n} \gamma^t R(s_t) \mid s_0 = s, \pi\right]$$

While, if no score increase (value > 0) can be obtained for any considered action sequence, the system instead plans for a future state of lowest state match value, whereby $(\forall k : (0 \leq k < n) \rightarrow S(s_k) = 1) \wedge S(s_n) < 1$, meaning the sequence is constrained to be planned in such a way that state match value is 1 except for the last action where it is minimized for the resulting state:

$$argmin_\pi S(s_n) < 1$$

This maximizes the agent's chance to reach the state of minimum state match while ensuring the low match value is not a consequence of predicting further from states where the knowledge was already not fully applicable.

4. **Predictor**: When the planning process queries the predicted state from a given state and an action, it constructs the predicted state by applying all knowledge to the given state in the following way: initializing with the cell values from the given state, for each cell we utilize only the rule r with $Q(r, c) = 1$ and maximum $f_{exp}(r)$, meaning the rule preconditions not only match perfectly to the given state and the action that has been considered, but also has the highest truth expectation among these rule candidates.

   In this case the postcondition cell value of the rule is applied to the corresponding cell at position (x, y) in the predicted state, while else the cell keeps the value from the previous state. Hence formally, for utilized rules

$$r^* = ((\overline{c}^1_t \wedge \overline{c}^2_t \wedge \overline{v}_t \wedge \overline{a}_t) \Rightarrow (\overline{c}^3_{t+1} \wedge \overline{v}_{t+1} \wedge R(r^*))) :$$

$$c_{t+1,x,y} = \begin{cases} c^3_{t+1} & \text{if } r^* = \underset{r \mid Q(r, c_{t,x,y}) = 1}{\text{argmax}} f_{\text{exp}}(r) \\ c_{t,x,y} & \text{otherwise} \end{cases}$$

   Now, while s_{t+1} is a composite of the cells at all locations at time $t + 1$ as calculated by the formula, the reward associated to s_{t+1} is the average of the reward of each of the N utilized rules:

$$R(s_{t+1}) = \frac{1}{N} \sum_{i=1}^{N} R(r^*_i)$$

   The system can hence be described by the pseudocode:

Algorithm 1: Pseudocode of NACE

- **Actual World:** perceived_array = perceive_partial(world)
- **Observer:**
  s_t = update_bird_view(s_{t-1}, perceived_array)
  $calculate(M_{change}, M^{observation}_{mismatched}, M^{prediction}_{mismatched})$
- **Hypothesizer:**
  - Create new rules for which $w_+(r) = 1$.
  - Update rule evidences according to $w_+(r)$ and $w_-(r)$.
  - Choose rules r_1 with $w_+(r_1) > w_-(r_1)$ for which there does not exist a rule r_2 with same precondition and action, but different postcondition with $f_{exp}(r_2) > f_{exp}(r_1)$.
- **Planner utilizing Predictor:**
  $a_1, ..., a_n = BFS\_with\_Predictor(V(s) > 0)$
  $a^*_1, ..., a^*_n = BFS\_with\_Predictor(min(S(s)) < 1)$
  //whereby BFS_with_Predictor is bounded breadth first search with Predictor as state transition function
  If found($a_1, ..., a_n$):, return $a_1, ..., a_n$
  If found($a^*_1, ..., a^*_n$):, return $a^*_1, ..., a^*_n$
  Else Perform a random action

5 Experiments

5.1 Minigrid environments

Minigrid (Chevalier-Boisvert et al. 2024) is a 2D grid world environment consisting of diverse procedurally generated

scenarios involving different types of objects, some of which can be manipulated or picked up by the agent. Depending on the level, the agent has to achieve different objectives and has to learn to achieve them based on observations provided by some observability model. Included observability models include full observability of the level, partial observability of grid cell values around the agent, and alternatively, full and partial observability of pixel values. For the purpose of comparisons in this paper, we chose partial observability of cell states around the agent.

We compared the techniques in the following Minigrid levels with partial observability, which by default do not depend on language input and color properties of involved objects. For further discussion we also put them into the following categories of environments, which impose different qualitative difficulties for learning agents:

1. Stationary environments where the start and goal location are the same in each episode.

2. Dynamic environments where start and goal location, as well as the location of obstacles, are randomized in each episode.

3. Dynamic environments with additional sequential dependencies beyond agent navigation (e.g. including a door that demands a key or switch to be used).

Environment	Category
MiniGrid-Empty-6x6-v0	1
MiniGrid-Empty-8x8-v0	1
MiniGrid-Empty-16x16-v0	1
MiniGrid-DistShift2-v0	1
MiniGrid-Empty-Random-5x5-v0	2
MiniGrid-Empty-Random-6x6-v0	2
BabyAI-GoToRedBallNoDists-v0	2
MiniGrid-LavaGapS7-v0	2
MiniGrid-SimpleCrossingS11N5-v0	2
MiniGrid-LavaCrossingS11N5-v0	2
MiniGrid-Unlock-v0	3
MiniGrid-DoorKey-8x8-v0	3
MiniGrid-UnlockPickup-v0	3
MiniGrid-BlockedUnlockPickup-v0	3

Table 1: Environments with corresponding values

5.2 Results

In each listed environment, the aforementioned techniques were compared with each other. For each environment, after every 100 time steps, we report the average reward, average episode length, and standard deviation from runs with five different environment seeds. In this section, we present and discuss the most representative results for each environment category, plots and tables of the remaining listed environments can be seen in the appendix.

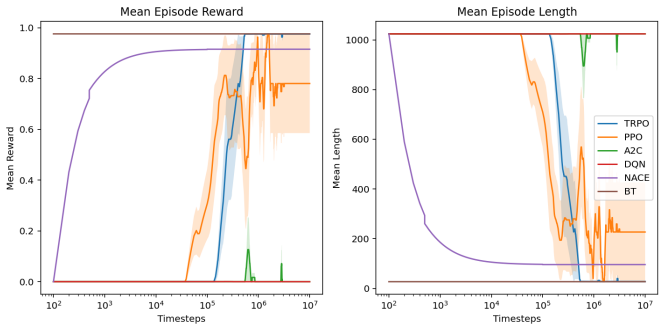

Figure 2: MiniGrid-Empty-16x16-v0

Table 2: Final values: MiniGrid-Empty-16x16-v0

Techn.	Reward	S.dev.	Ep. len.	S. dev.
TRPO	**0.976**	**0.001**	**27.600**	**1.200**
PPO	0.781	0.390	226.800	398.600
A2C	0.000	0.000	1024.000	0.000
DQN	0.000	0.000	1024.000	0.000
NACE	0.916	0.004	95.800	4.261

In MiniGrid-Empty-16x16-v0 (as in Figure 2, and table 2), there is no variability in the level as the start and goal locations do not vary. In this case, particularly the NACE agent follows its innate strategy, e.g. first learning to move the agent via the actions, then hitting observed walls out of curiosity, and finally exploring initially invisible parts of the map until the goal object is found and moved into. The primary difficulty in this level is coming from the large map with a limited observation window and from the sparse reward: only moving to the goal location is rewarding. NACE, due to its intrinsic inductive bias, within less than 10^3 time steps, explores the area systematically and associates reward with the goal location. Hence it is able to robustly find and move to the goal location. However, on the DRL side, DQN and A2C fail to learn even with this complexity level of the task. Additionally, PPO and TRPO converge 1000 times slower, requiring 10 million time steps to reach an effective policy, with TRPO being as effective as BT, followed by NACE and PPO.

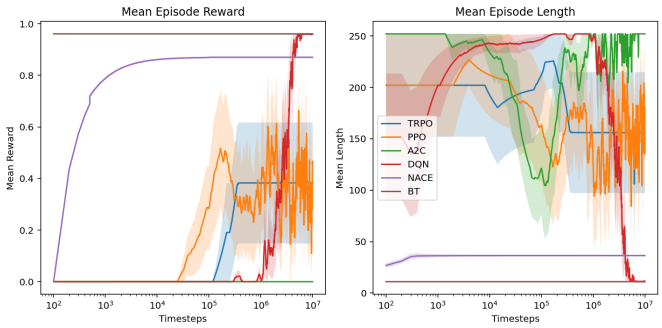

Figure 3: MiniGrid-DistShift2-v0

Table 3: Final values: MiniGrid-DistShift2-v0

Techn.	Reward	S.dev.	Ep. len.	S. dev.
TRPO	0.383	0.469	156.000	117.577
PPO	0.763	0.381	60.800	95.608
A2C	0.000	0.000	252.000	0.000
DQN	**0.961**	**0.000**	**11.000**	**0.000**
NACE	0.870	0.006	36.400	1.744

In MiniGrid-DistShift2-v0 (as in Figure 3 and table 3), the start and goal location as well as position of lava obstacles does not change. DQN reached a policy (average reward 0.961) close to the Behavior Tree, while NACE is slightly worse (0.87), and the policies found by PPO and TRPO were even more sub-optimal (0.763 and 0.383 respectively). A2C on the other hand did not converge to a policy with an average reward above 0.

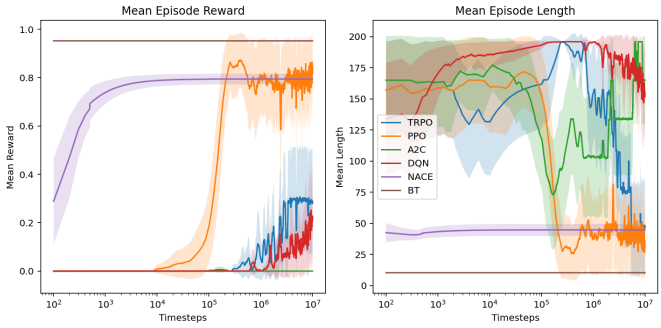

Figure 4: MiniGrid-LavaGapS7-v0

Table 4: Final values: MiniGrid-LavaGapS7-v0

Techn.	Reward	S.dev.	Ep. len.	S. dev.
TRPO	0.187	0.375	74.280	91.398
PPO	**0.838**	**0.309**	**32.760**	**60.285**
A2C	0.000	0.000	165.000	71.031
DQN	0.114	0.309	173.800	60.119
NACE	0.794	0.044	44.800	9.558

In MiniGrid-LavaGapS7-v0 (as in Figure 4, and table 4), the reward and episode lengths do not correspond well to each other, as the episode resets when the agent hits the lava. However, from the mean episode reward, it is clear that PPO and NACE find a similarly effective strategy, whereby NACE takes about 10^3 time steps while PPO takes 3×10^5 time steps to get to mean reward close to 0.8, while the optimal policies as BT shows can be between 0.9 and 1.0. Additionally, PPO exhibits more instability in learning and greater sensitivity to initialization, as evidenced by a higher standard deviation. DQN and TRPO performance was surprisingly poor (even A2C completely fails to learn any policy), as this level is practically fully observable: there is only a 5x5 free space area to explore by the agent without any walls, which is mostly covered by the default observation window. This means that to perform well in this sub-domain, the agent does not need to explore the area systematically, nor does it need to remember prior locations. The challenge here is to avoid the lava, which differently than in MiniGrid-DistShift2-v0, spawns at different locations, while reaching the goal location, which (together with the start location) is always the same.

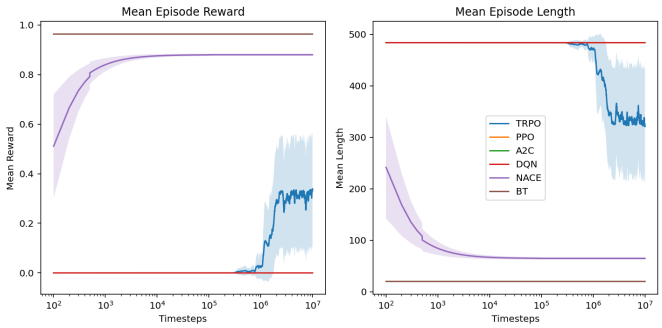

Figure 5: MiniGrid-SimpleCrossingS11N5-v0

Table 5: Final values: MiniGrid-SimpleCrossingS11N5-v0

Techn.	Reward	S.dev.	Ep. len.	S. dev.
TRPO	0.381	0.467	300.480	224.786
PPO	0.000	0.000	484.000	0.000
A2C	0.000	0.000	484.000	0.000
DQN	0.000	0.000	484.000	0.000
NACE	**0.880**	**0.009**	**64.800**	**4.665**

In MiniGrid-SimpleCrossingS11N5-v0 (as in Figure 5, and table 5) the inductive bias of NACE to remember observations outside of the perceived window is particularly effective compared to the other techniques, TRPO was able to find a sub-optimal policy with significant less average reward, while the other techniques did not converge to a working policy. In future work we will also analyze how much more data is required for the model to learn useful state tracking ability, but as seen, with the used techniques it did not happen within the 10^7 time steps.

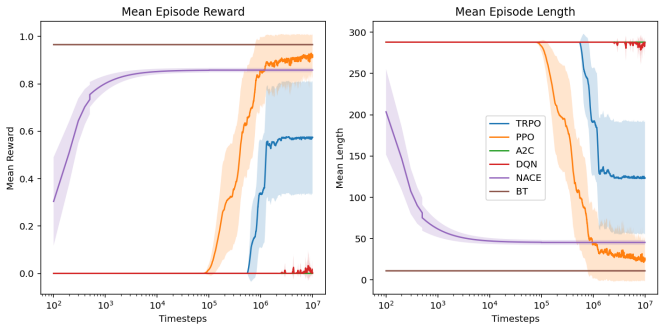

Figure 6: MiniGrid-Unlock-v0

Table 6: Final values: MiniGrid-Unlock-v0

Techn.	Reward	S.dev.	Ep. len.	S. dev.
TRPO	0.577	0.471	122.520	135.143
PPO	**0.890**	**0.263**	**32.480**	**75.380**
A2C	0.000	0.000	288.000	0.000
DQN	0.000	0.000	288.000	0.000
NACE	0.858	0.018	45.400	5.817

In MiniGrid-Unlock-v0 (as in Figure 6, and table 6), the primary difficulty is a sequential dependency between the picking up of the key before the door can be entered to obtain the reward. Even though it is a single sequential dependency, the technique that performed best in our runs, PPO, demands about a million time steps to converge to a similarly effective policy as NACE (again within 10^3 steps). Additionally, PPO shows more instability in learning. As also visible, in our runs, TRPO did not exceed a mean episode reward of 0.6. Moreover, A2C and DQN completely fail to learn any effective policy.

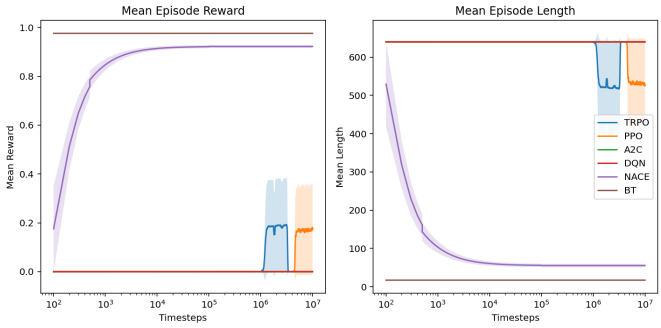

Figure 7: MiniGrid-DoorKey-8x8-v0

Table 7: Final values: MiniGrid-DoorKey-8x8-v0

Techn.	Reward	S.dev.	Ep. len.	S. dev.
TRPO	0.000	0.000	640.000	0.000
PPO	0.156	0.357	540.520	227.947
A2C	0.000	0.000	640.000	0.000
DQN	0.000	0.000	640.000	0.000
NACE	**0.922**	**0.012**	**55.400**	**8.777**

Finally, in MiniGrid-DoorKey-8x8-v0 (as in Figure 7, and table 7), we could not make any of the DRL techniques converge to an effective policy within the 10 million time steps. This is despite the fact that the only additional complexity lies in going through the opened door to reach a goal location in the other room, which essentially adds one sequential dependency and makes the reward more sparse, as no reward is associated with using the door by itself. This suggests bad combinatorial scaling of the involved DRL techniques, while NACE, on average, needs a comparatively similar amount of time as for MiniGrid-Unlock-v0.

These four representative results of each environment category highlight how DRL techniques are failing to learn grid world tasks data-efficiently as they become slightly more complicated and how they are sensitive at this level. Part of it can be explained by lacking the ability to establish (to learn and explicitly represent) causal relationships in tasks, and missing inductive biases that are beneficiary in grid worlds.

With the environments in the final values tables and table 9 NACE reaches the final average rewards after just 1000 training steps, whereby the value for the other Reinforcement Learning techniques is 0 at that time step, as they tend to require at least hundreds of thousands of training steps for effective policies to be found. This is also consistent with the literature and more specialized methods with intrinsic rewards for faster learning (as evidenced by (Chen and Guan 2023) and (Guo et al. 2022)), which results we summarized in terms of average reward and training iterations in table 8.

Environment	Reward	Steps	Algo
MG-DoorKey-16x16-v0	0.61	3E+06	CEMP
MG-DoorKey-16x16-v0	**0.87**	1E+04	NACE
MG-DoorKey-8x8-v0	**0.98**	3E+06	CEMP
MG-DoorKey-8x8-v0	0.39	2E+05	CEMP
MG-DoorKey-8x8-v0	0.92	1E+03	NACE
MG-LavaCrossingS11N5-v0	0.81	3E+06	CEMP
MG-LavaCrossingS11N5-v0	0.01	2E+05	CEMP
MG-LavaCrossingS11N5-v0	**0.88**	1E+03	NACE
MG-UnlockPickup-v0	**0.95**	3E+06	CEMP
MG-UnlockPickup-v0	0.20	3E+06	CEMP
MG-UnlockPickup-v0	0.86	1E+03	NACE
MG-BlockedUnlockPickup-v0	0.68	1E+07	ViewX
MG-BlockedUnlockPickup-v0	**0.89**	1E+03	NACE
MG-SimpleCrossingS9N2-v0	0.21	1E+05	Graph
MG-SimpleCrossingS9N2-v0	**0.85**	1E+03	NACE

Table 8: Performance of NACE and different algorithms on various MiniGrid (MG) environments according to (Chen and Guan 2023), (Guo et al. 2022) and (Zhang et al. 2021)

6 Conclusion

We have outlined the strengths and weaknesses between standard Deep RL techniques and our NACE algorithm. While the former usually gets close to optimal policies when enough training data exists, the necessary amount can considerably grow based on task complexities such as additional sequential dependencies. Hence, we think our contribution NACE represents a quite natural extension of the Reinforcement Learning framework to support empirical causal relations the agent can use to perform effective learning and decision making even in low data regime, and without depending on a pre-defined causal model. As we have shown, our causality-informed curiosity model, together with the outlined inductive biases, establishes systematic exploration in grid worlds with high data efficiency. Additionally, our analysis of inductive biases in grid world environments can shed light on alternative learning mechanisms which could perform well in such domains. In future work we will attempt to generalize the technique to handle three-dimensional and continuous spaces, and neural implementations of NACE for causality-informed Deep RL will be explored too.

7 Acknowledgements

Thank you to Adrian Borucki for his valuable comments to improve the formalisms in the paper, as well as Robert Wünsche for his suggestions.

8 Appendix

Table 9: All final values

Level	Techn.	Reward	S.dev.	Ep. len.	S. dev.
BabyAI-GoToRedBallNoDists-v0	**TRPO**	0.906	0.056	6.680	3.947
BabyAI-GoToRedBallNoDists-v0	**PPO**	**0.922**	**0.036**	**5.520**	**2.547**
BabyAI-GoToRedBallNoDists-v0	**A2C**	0.039	0.190	61.520	12.149
BabyAI-GoToRedBallNoDists-v0	**DQN**	0.411	0.464	37.920	29.459
BabyAI-GoToRedBallNoDists-v0	**NACE**	0.876	0.038	35.200	10.685
MiniGrid-DistShift2-v0	**TRPO**	0.383	0.469	156.000	117.577
MiniGrid-DistShift2-v0	**PPO**	0.763	0.381	60.800	95.608
MiniGrid-DistShift2-v0	**A2C**	0.000	0.000	252.000	0.000
MiniGrid-DistShift2-v0	**DQN**	**0.961**	**0.000**	**11.000**	**0.000**
MiniGrid-DistShift2-v0	**NACE**	0.870	0.006	36.400	1.744
MiniGrid-DoorKey-8x8-v0	**TRPO**	0.000	0.000	640.000	0.000
MiniGrid-DoorKey-8x8-v0	**PPO**	0.156	0.357	540.520	227.947
MiniGrid-DoorKey-8x8-v0	**A2C**	0.000	0.000	640.000	0.000
MiniGrid-DoorKey-8x8-v0	**DQN**	0.000	0.000	640.000	0.000
MiniGrid-DoorKey-8x8-v0	**NACE**	**0.922**	**0.012**	**55.400**	**8.777**
MiniGrid-Empty-16x16-v0	**TRPO**	**0.976**	**0.001**	**27.600**	**1.200**
MiniGrid-Empty-16x16-v0	**PPO**	0.781	0.390	226.800	398.600
MiniGrid-Empty-16x16-v0	**A2C**	0.000	0.000	1024.000	0.000
MiniGrid-Empty-16x16-v0	**DQN**	0.000	0.000	1024.000	0.000
MiniGrid-Empty-16x16-v0	**NACE**	0.916	0.004	95.800	4.261
MiniGrid-Empty-6x6-v0	**TRPO**	**0.956**	**0.000**	**7.000**	**0.000**
MiniGrid-Empty-6x6-v0	**PPO**	0.190	0.380	116.800	54.400
MiniGrid-Empty-6x6-v0	**A2C**	0.000	0.000	144.000	0.000
MiniGrid-Empty-6x6-v0	**DQN**	0.764	0.382	34.600	54.701
MiniGrid-Empty-6x6-v0	**NACE**	0.843	0.014	25.200	2.227
MiniGrid-Empty-8x8-v0	**TRPO**	**0.961**	**0.000**	**11.000**	**0.000**
MiniGrid-Empty-8x8-v0	**PPO**	0.190	0.380	207.600	96.800
MiniGrid-Empty-8x8-v0	**A2C**	0.000	0.000	256.000	0.000
MiniGrid-Empty-8x8-v0	**DQN**	0.956	0.002	12.600	0.490
MiniGrid-Empty-8x8-v0	**NACE**	0.846	0.010	43.800	2.713
MiniGrid-Empty-Random-5x5-v0	**TRPO**	0.961	0.013	4.320	1.406
MiniGrid-Empty-Random-5x5-v0	**PPO**	0.967	0.011	3.640	1.229
MiniGrid-Empty-Random-5x5-v0	**A2C**	0.159	0.363	84.160	36.294
MiniGrid-Empty-Random-5x5-v0	**DQN**	**0.972**	**0.013**	**3.120**	**1.423**
MiniGrid-Empty-Random-5x5-v0	**NACE**	0.833	0.044	18.600	4.841
MiniGrid-Empty-Random-6x6-v0	**TRPO**	0.971	0.011	4.560	1.699
MiniGrid-Empty-Random-6x6-v0	**PPO**	**0.974**	**0.009**	**4.080**	**1.495**
MiniGrid-Empty-Random-6x6-v0	**A2C**	0.040	0.194	138.320	27.826
MiniGrid-Empty-Random-6x6-v0	**DQN**	0.860	0.318	20.480	45.638
MiniGrid-Empty-Random-6x6-v0	**NACE**	0.835	0.066	26.400	10.538
MiniGrid-LavaCrossingS11N5-v0	**TRPO**	0.000	0.000	291.120	236.230
MiniGrid-LavaCrossingS11N5-v0	**PPO**	0.000	0.000	426.200	156.524
MiniGrid-LavaCrossingS11N5-v0	**A2C**	0.000	0.000	291.520	235.743
MiniGrid-LavaCrossingS11N5-v0	**DQN**	0.000	0.000	484.000	0.000
MiniGrid-LavaCrossingS11N5-v0	**NACE**	**0.875**	**0.004**	**67.000**	**2.191**
MiniGrid-LavaGapS7-v0	**TRPO**	0.187	0.375	74.280	91.398
MiniGrid-LavaGapS7-v0	**PPO**	**0.838**	**0.309**	**32.760**	**60.285**
MiniGrid-LavaGapS7-v0	**A2C**	0.000	0.000	165.000	71.031
MiniGrid-LavaGapS7-v0	**DQN**	0.114	0.309	173.800	60.119
MiniGrid-LavaGapS7-v0	**NACE**	0.794	0.044	44.800	9.558
MiniGrid-SimpleCrossingS11N5-v0	**TRPO**	0.381	0.467	300.480	224.786
MiniGrid-SimpleCrossingS11N5-v0	**PPO**	0.000	0.000	484.000	0.000
MiniGrid-SimpleCrossingS11N5-v0	**A2C**	0.000	0.000	484.000	0.000
MiniGrid-SimpleCrossingS11N5-v0	**DQN**	0.000	0.000	484.000	0.000
MiniGrid-SimpleCrossingS11N5-v0	**NACE**	**0.880**	**0.009**	**64.800**	**4.665**
MiniGrid-Unlock-v0	**TRPO**	0.577	0.471	122.520	135.143
MiniGrid-Unlock-v0	**PPO**	**0.890**	**0.263**	**32.480**	**75.380**
MiniGrid-Unlock-v0	**A2C**	0.000	0.000	288.000	0.000
MiniGrid-Unlock-v0	**DQN**	0.000	0.000	288.000	0.000
MiniGrid-Unlock-v0	**NACE**	0.858	0.018	45.400	5.817

Table 10: All values after 1K steps

Level	Techn.	Reward	S.dev.	Ep. len.	S. dev.
BabyAI-GoToRedBallNoDists-v0	**TRPO**	0.000	0.000	64.000	0.000
BabyAI-GoToRedBallNoDists-v0	**PPO**	0.000	0.000	64.000	0.000
BabyAI-GoToRedBallNoDists-v0	**A2C**	0.000	0.000	64.000	0.000
BabyAI-GoToRedBallNoDists-v0	**DQN**	0.000	0.000	64.000	0.000
BabyAI-GoToRedBallNoDists-v0	**NACE**	**0.876**	**0.038**	**35.200**	**10.685**
MiniGrid-DistShift2-v0	**TRPO**	0.000	0.000	202.000	100.000
MiniGrid-DistShift2-v0	**PPO**	0.000	0.000	202.000	100.000
MiniGrid-DistShift2-v0	**A2C**	0.000	0.000	252.000	0.000
MiniGrid-DistShift2-v0	**DQN**	0.000	0.000	252.000	0.000
MiniGrid-DistShift2-v0	**NACE**	**0.870**	**0.006**	**36.400**	**1.744**
MiniGrid-DoorKey-8x8-v0	**TRPO**	0.000	0.000	640.000	0.000
MiniGrid-DoorKey-8x8-v0	**PPO**	0.000	0.000	640.000	0.000
MiniGrid-DoorKey-8x8-v0	**A2C**	0.000	0.000	640.000	0.000
MiniGrid-DoorKey-8x8-v0	**DQN**	0.000	0.000	640.000	0.000
MiniGrid-DoorKey-8x8-v0	**NACE**	**0.922**	**0.012**	**55.400**	**8.777**
MiniGrid-Empty-16x16-v0	**TRPO**	0.000	0.000	1024.000	0.000
MiniGrid-Empty-16x16-v0	**PPO**	0.000	0.000	1024.000	0.000
MiniGrid-Empty-16x16-v0	**A2C**	0.000	0.000	1024.000	0.000
MiniGrid-Empty-16x16-v0	**DQN**	0.000	0.000	1024.000	0.000
MiniGrid-Empty-16x16-v0	**NACE**	**0.916**	**0.004**	**95.800**	**4.261**
MiniGrid-Empty-6x6-v0	**TRPO**	0.000	0.000	144.000	0.000
MiniGrid-Empty-6x6-v0	**PPO**	0.000	0.000	144.000	0.000
MiniGrid-Empty-6x6-v0	**A2C**	0.000	0.000	144.000	0.000
MiniGrid-Empty-6x6-v0	**DQN**	0.000	0.000	144.000	0.000
MiniGrid-Empty-6x6-v0	**NACE**	**0.843**	**0.014**	**25.200**	**2.227**
MiniGrid-Empty-8x8-v0	**TRPO**	0.000	0.000	256.000	0.000
MiniGrid-Empty-8x8-v0	**PPO**	0.000	0.000	256.000	0.000
MiniGrid-Empty-8x8-v0	**A2C**	0.000	0.000	256.000	0.000
MiniGrid-Empty-8x8-v0	**DQN**	0.000	0.000	256.000	0.000
MiniGrid-Empty-8x8-v0	**NACE**	**0.846**	**0.010**	**43.800**	**2.713**
MiniGrid-Empty-Random-5x5-v0	**TRPO**	0.039	0.192	96.080	19.204
MiniGrid-Empty-Random-5x5-v0	**PPO**	0.039	0.192	96.080	19.204
MiniGrid-Empty-Random-5x5-v0	**A2C**	0.157	0.360	84.320	35.927
MiniGrid-Empty-Random-5x5-v0	**DQN**	0.157	0.360	84.320	35.927
MiniGrid-Empty-Random-5x5-v0	**NACE**	**0.833**	**0.044**	**18.600**	**4.841**
MiniGrid-Empty-Random-6x6-v0	**TRPO**	0.000	0.000	144.000	0.000
MiniGrid-Empty-Random-6x6-v0	**PPO**	0.000	0.000	144.000	0.000
MiniGrid-Empty-Random-6x6-v0	**A2C**	0.118	0.320	127.040	45.928
MiniGrid-Empty-Random-6x6-v0	**DQN**	0.118	0.320	127.040	45.928
MiniGrid-Empty-Random-6x6-v0	**NACE**	**0.835**	**0.066**	**26.400**	**10.538**
MiniGrid-LavaCrossingS11N5-v0	**TRPO**	0.000	0.000	368.640	205.288
MiniGrid-LavaCrossingS11N5-v0	**PPO**	0.000	0.000	368.640	205.288
MiniGrid-LavaCrossingS11N5-v0	**A2C**	0.000	0.000	310.360	231.521
MiniGrid-LavaCrossingS11N5-v0	**DQN**	0.000	0.000	484.000	0.000
MiniGrid-LavaCrossingS11N5-v0	**NACE**	**0.875**	**0.004**	**67.000**	**2.191**
MiniGrid-LavaGapS7-v0	**TRPO**	0.000	0.000	164.880	71.305
MiniGrid-LavaGapS7-v0	**PPO**	0.000	0.000	164.880	71.305
MiniGrid-LavaGapS7-v0	**A2C**	0.000	0.000	164.960	71.123
MiniGrid-LavaGapS7-v0	**DQN**	0.000	0.000	196.000	0.000
MiniGrid-LavaGapS7-v0	**NACE**	**0.794**	**0.044**	**44.800**	**9.558**
MiniGrid-SimpleCrossingS11N5-v0	**TRPO**	0.000	0.000	484.000	0.000
MiniGrid-SimpleCrossingS11N5-v0	**PPO**	0.000	0.000	484.000	0.000
MiniGrid-SimpleCrossingS11N5-v0	**A2C**	0.000	0.000	484.000	0.000
MiniGrid-SimpleCrossingS11N5-v0	**DQN**	0.000	0.000	484.000	0.000
MiniGrid-SimpleCrossingS11N5-v0	**NACE**	**0.880**	**0.009**	**64.800**	**4.665**
MiniGrid-Unlock-v0	**TRPO**	0.000	0.000	288.000	0.000
MiniGrid-Unlock-v0	**PPO**	0.000	0.000	288.000	0.000
MiniGrid-Unlock-v0	**A2C**	0.000	0.000	288.000	0.000
MiniGrid-Unlock-v0	**DQN**	0.000	0.000	288.000	0.000
MiniGrid-Unlock-v0	**NACE**	**0.858**	**0.018**	**45.400**	**5.817**

References

Bryce, D. 2011. Wumpus World in introductory artificial intelligence. *Journal of Computing Sciences in Colleges*, 27(2): 58–65.

Burda, Y.; Edwards, H.; Storkey, A.; and Klimov, O. 2018. Exploration by random network distillation. *arXiv preprint arXiv:1810.12894*.

Chen, Z.; and Guan, Q. 2023. Continuous Exploration via Multiple Perspectives in Sparse Reward Environment. In *Chinese Conference on Pattern Recognition and Computer Vision (PRCV)*, 57–68. Springer.

Chevalier-Boisvert, M.; Dai, B.; Towers, M.; Perez-Vicente, R.; Willems, L.; Lahlou, S.; Pal, S.; Castro, P. S.; and Terry, J. 2024. Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks. *Advances in Neural Information Processing Systems*, 36.

Colledanchise, M.; and Ögren, P. 2018. *Behavior trees in robotics and AI: An introduction*. CRC Press.

Cook, B.; and Hammer, P. 2024. Autonomous Intelligent Reinforcement Inferred Symbolism. In *International Conference on Artificial General Intelligence*, 53–62. Springer.

Guo, H.; Wu, F.; Qin, Y.; Li, R.; Li, K.; and Li, K. 2023. Recent trends in task and motion planning for robotics: A survey. *ACM Computing Surveys*, 55(13s): 1–36.

Guo, Y.; Fu, Y.; Peng, R.; and Lee, H. 2022. Learning Exploration Policies with View-based Intrinsic Rewards. In *Deep Reinforcement Learning Workshop NeurIPS 2022*.

Mnih, V.; Badia, A. P.; Mirza, M.; Graves, A.; Lillicrap, T.; Harley, T.; Silver, D.; and Kavukcuoglu, K. 2016. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*, 1928–1937. PMLR.

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Graves, A.; Antonoglou, I.; Wierstra, D.; and Riedmiller, M. 2013. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*.

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; et al. 2015. Human-level control through deep reinforcement learning. *nature*, 518(7540): 529–533.

Pearl, J. 1995. From Bayesian networks to causal networks. In *Mathematical models for handling partial knowledge in artificial intelligence*, 157–182. Springer.

Pollack, M. E.; and Ringuette, M. 1990. Introducing the Tileworld: Experimentally evaluating agent architectures. In *AAAI*, volume 90, 183–189.

Schulman, J.; Levine, S.; Abbeel, P.; Jordan, M.; and Moritz, P. 2015. Trust region policy optimization. In *International conference on machine learning*, 1889–1897. PMLR.

Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.

Spaan, M. T. 2012. Partially observable Markov decision processes. *Reinforcement learning: State-of-the-art*, 387–414.

Wang, P. 2013. *Non-axiomatic logic: A model of intelligent reasoning*. World Scientific.

Zhang, T.; Kirk, R.; Rocktäschel, T.; Grefenstette, E.; et al. 2021. Graph Backup: Data Efficient Backup Exploiting Markovian Data. In *Deep RL Workshop NeurIPS 2021*.

Zhang, T.; Xu, H.; Wang, X.; Wu, Y.; Keutzer, K.; Gonzalez, J. E.; and Tian, Y. 2020. Bebold: Exploration beyond the boundary of explored regions. *arXiv preprint arXiv:2012.08621*.

Zheng, L.; Chen, J.; Wang, J.; He, J.; Hu, Y.; Chen, Y.; Fan, C.; Gao, Y.; and Zhang, C. 2021. Episodic multi-agent reinforcement learning with curiosity-driven exploration. *Advances in Neural Information Processing Systems*, 34: 3757–3769.

Zheng, X.; Aragam, B.; Ravikumar, P. K.; and Xing, E. P. 2018. Dags with no tears: Continuous optimization for structure learning. *Advances in neural information processing systems*, 31.